

4-1 · MinIO 오브젝트 스토리지 구축 · 실습 1.5H

# MinIO 컨테이너 구축

MinIO 를 띄우고, 안과 밖의 주소 차이를 눈으로

수도권ICT 이노베이션스퀘어  
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

# 1.5시간 뒤에는

---

- ① **docker-compose.yml 로 minio·mc 두 컨테이너를 Up 상태로 띄운다**
- ② **웹 콘솔(9001)에 minioadmin 으로 로그인해 메뉴를 둘러본다**
- ③ **컨테이너 안에서 minio:9000 은 되고 localhost:9000 은 안 됨을 본다**
- ④ **볼륨이 down·up 뒤에도 남는지, 로그를 어디서 보는지 안다**

**오늘 만든 compose 파일은 4주차 내내 확장합니다 — ~/lakehouse 를 지우지 마세요**

# 다섯 단계 — ~/lakehouse 한 폴더에서 끝까지

~/lakehouse 폴더를 새로 만들고 시작합니다 — 1-5 에서 배운 Compose 문법이 그대로 쓰입니다

## 1. 폴더·Compose

~/lakehouse  
compose 파일

## 2. 웹 콘솔

localhost:9001  
둘러보기

## 3. 네트워크

minio:9000 vs  
localhost:9000

## 4. 볼륨

down → up  
minio-data 확인

## 5. 로그

compose logs  
배너 확인

# docker-compose.yml — 설정 한 줄씩 뜻

| 설정                 | 뜻                         |
|--------------------|---------------------------|
| minio: image       | pgsty/minio — 서버          |
| ports              | 9000 S3 API · 9001 웹 콘솔   |
| environment        | 루트 계정 minioadmin (실습 전용)  |
| command            | server /data 콘솔 :9001     |
| volumes minio-data | 이름 볼륨 — 지워도 데이터가 남는다      |
| mc: image          | pgsty/mc — 클라이언트          |
| mc: entrypoint     | sleep infinity — 계속 떠 있게  |
| mc: volumes        | ./data:/data — 호스트 폴더 마운트 |

mc 는 명령만 실행하고 끝나는 이미지 — sleep infinity 로 계속 떠 있게 만드는 흔한 수법

## 1단계

# 핵심 명령 — 만들고 · 검증 · 띄우고 · 확인

---

|    |                                                                  |
|----|------------------------------------------------------------------|
| 폴더 | <code>mkdir -p ~/lakehouse/data &amp;&amp; cd ~/lakehouse</code> |
| 검증 | <code>docker compose config</code>                               |
| 기동 | <code>docker compose up -d</code>                                |
| 상태 | <code>docker compose ps</code>                                   |
| 콘솔 | <code>http://localhost:9001 → minioadmin / minioadmin</code>     |

확인 포인트: ps 에 minio·mc 두 컨테이너가 Up, 브라우저에 MinIO 콘솔 로그인 화면이 뜬다

# 웹 콘솔(9001) 둘러보기 — 메뉴 세 곳

**1 Object Browser**  
버킷과 오브젝트를 보는 곳 — 지금은 비어 있다

**2 Monitoring → Metrics**  
용량과 요청 수

**3 Access Keys**  
프로그램용 키를 따로 만드는 곳(심화 과제)

**4 9000 과 9001**  
9000 은 프로그램이 쓰는 S3 API, 9001 은 사람이 보는 웹 콘솔

minioadmin 은 기본 계정이라 실습에서만 — 운영은 반드시 바꾸고, 프로그램엔 권한을 제한한 액세스 키를 발급

# 컨테이너 안의 localhost — 실패해야 정상

1-5 의 「밖에서는 localhost, 안에서는 서비스 이름」— 오늘 가장 많이 만날 오류의 원인입니다

## ① 호스트에서

```
curl localhost:9000  
→ 200
```

## ② mc 안에서

```
wget minio:9000  
→ 성공
```

## ③ mc 안 localhost

```
wget localhost:9000  
→ 실패 = 정상
```

# 같은 9000 인데 — 왜 하나는 되고 하나는 안 되나

## minio:9000 — 서비스 이름

컨테이너 네트워크 안에서 이름으로 찾는다



VS

## localhost:9000 — 자기 자신

컨테이너 안의 localhost 엔 MinIO 가 없다



# 볼륨이 데이터를 지키는지, 로그는 어디서 보는지

1

## 볼륨이 붙어 있는지

`docker volume ls | grep minio · inspect 에 minio-data → /data`

2

## 지웠다 다시 만들어도

`docker compose down → up -d → ps` — 볼륨 목록에 minio-data 그대로

3

## 로그를 읽는 법

`docker compose logs minio | tail -n 20 · -f` 는 Ctrl+C 로 빠져나온다

로그에서 찾을 것

서버 배너

API 주소

# 이 1.5시간이 끝날 때 확인할 것

- 1 docker compose ps 에 minio·mc 두 컨테이너가 Up
- 2 http://localhost:9001 에 minioadmin 으로 로그인된다
- 3 호스트 200 · 안에서 minio:9000 성공 · localhost:9000 실패
- 4 inspect 에 minio-data → /data, down·up 뒤에도 볼륨이 남는다
- 5 logs 에 「MinIO Object Storage Server」 배너와 API·Console 주소

막혔을 때 순서는 언제나 같다 — ps → logs 서비스명 → 포트 충돌 확인

4-1 · 실습 1.5H 마무리

## 다음은 mc CLI 와 S3 API 활용 (1H)

버킷을 만들고 파일을 올리고, 파이썬 boto3 로도 같은 저장소를 다뤄 봅니다

✓ mc alias set → mc mb → mc ls

✓ mc cp 로 올리고 mc cat 으로 읽기

✓ 슬래시 키로 올려 폴더가 아님을 확인

✓ boto3 — endpoint\_url 만 바꾸면 AWS

가져갈 것

minio·mc 가 Up 인 ~/lakehouse